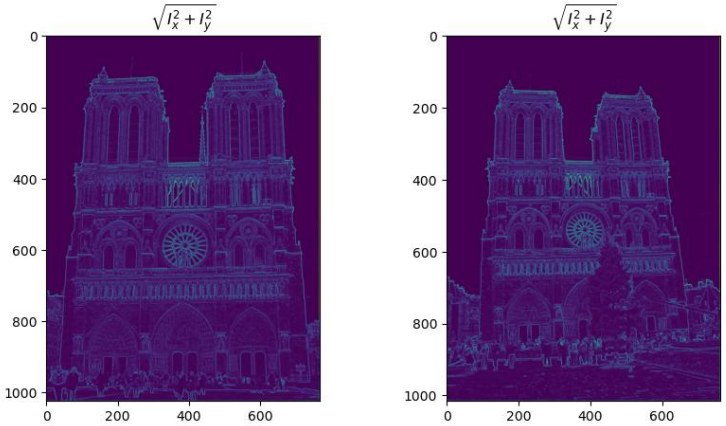
Warning: Do not delete slides.
This includes extra credit slides and any problems you do not complete.
All problems, including extra credit, must be assigned to a slide on Gradescope. Failure to follow this will result in a penalty.

CS 6476 Project 2

Nandan Bharatkumar Parikh
nparikh44@gatech.edu
nparikh44
903487850

Part 1: Harris corner detector

[i
[



$$\sqrt{\left(I_x^2 + I_y^2\right)}$$

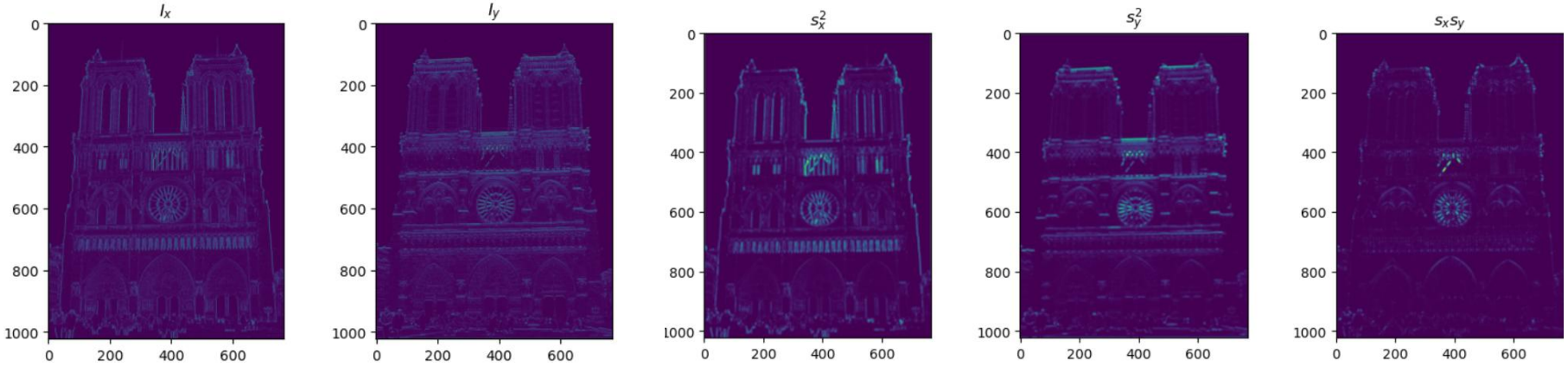
The above represent gradient magnitude at a pixel (x, y). This value is important to the Harris corner detector because corners are points where gradients change significantly in both direction

Corners/Junctions: Pixels where two or more edges meet have large changes in intensity in both directions because the magnitude in both directions is quite strong

Edges: Edges will also have high magnitudes since the gradient changes sharply in a particular direction (or both if the edge is diagonal)

For the Notre Dame image, the highest magnitudes are at architectural edges and detailed regions like stone carvings and decorative patterns

Part 1: Harris corner detector



Ix, Iy, Sx, Sy, Sxy measure how image intensity changes locally:
Ix, Iy – intensity changes along x and y directions
Sx = Ix², Sy = Iy², Sxy = Ix·Iy – capture combined variations and correlations

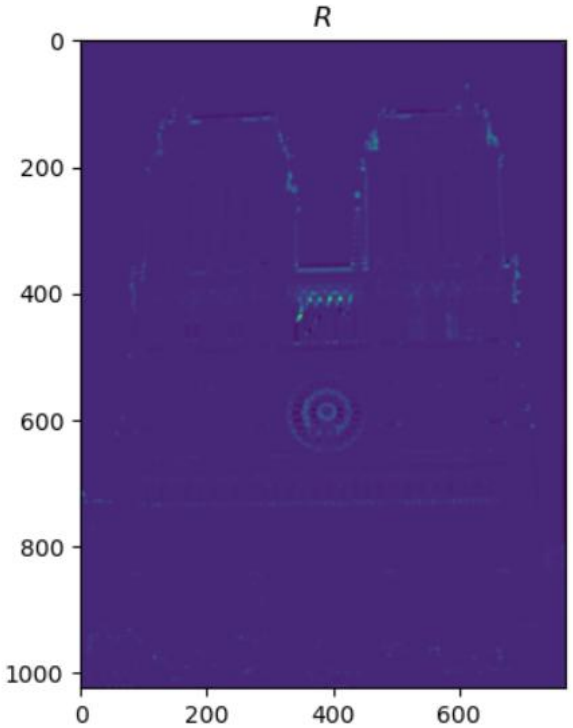
These values are used to build the **structure tensor**, which identifies corners as points where intensity changes strongly in both directions

Part 1: Harris corner detector

Why we convolve with a Gaussian:

- Smooths the image to **reduce the effect of noise**, which could otherwise create false corners
- Aggregates gradient information over a **small neighborhood**, capturing consistent local intensity changes
- Gives **more weight to pixels near the center**, preserving the local structure while still smoothing
- Helps the Harris detector focus on **meaningful, stable corners** instead of tiny texture variations or random intensity spikes

Part 1: Harris corner detector



Additive Shifts:

- Gradients are immune to additive shifts since they measure the **difference between neighboring pixels** so a uniform shift will cancel out

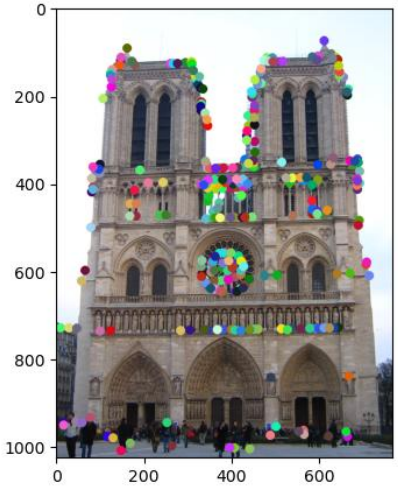
Multiplicative Gain:

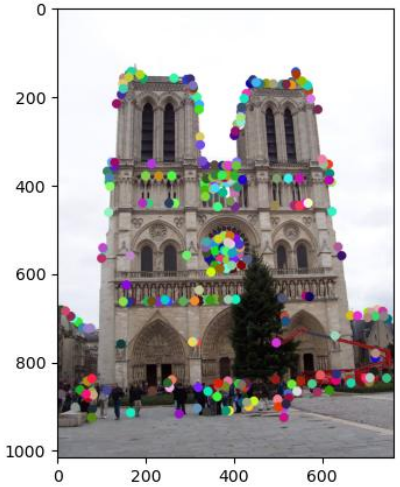
- If we multiply all the pixels by a constant “k” then the gradients are scaled by “k”

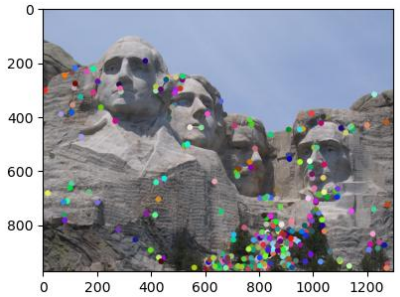
$$I'_x = \frac{\partial kI}{\partial x} = k\frac{\partial I}{\partial x}$$

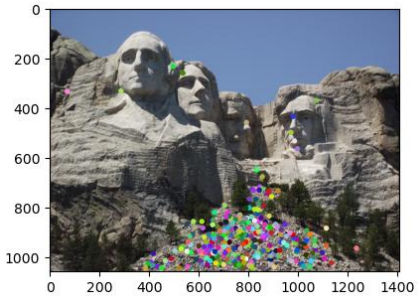
- Hence the gradient features are **not invariant** to multiplicative gain

Part 1: Harris corner detector

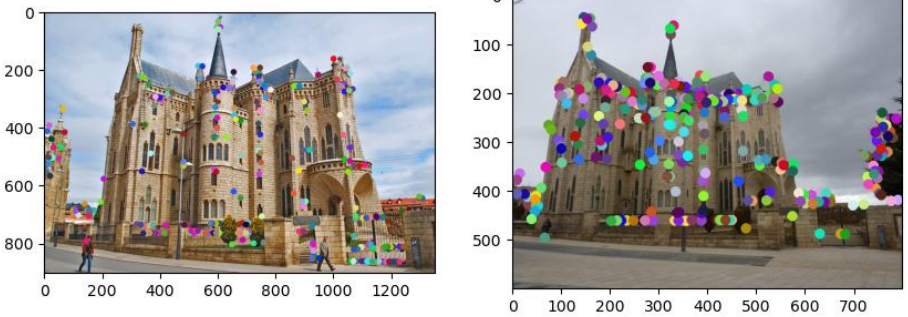








Part 1: Harris corner detector



Disadvantages:

- Dependent on window size and value of k. If the window is too large, we may miss corners if they are close together. If the window is too small, we may interpret noise as a corner

Advantages:

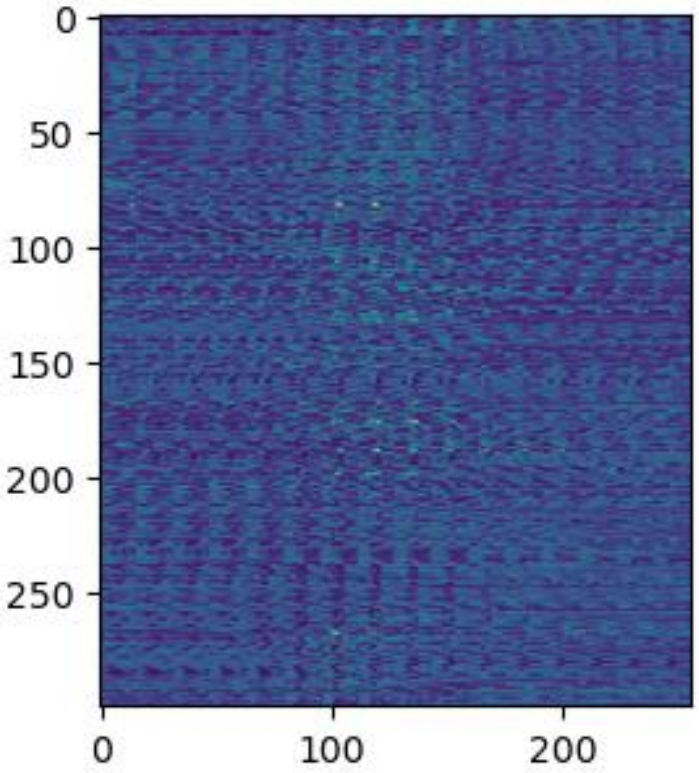
- It is simple to compute and parallelise on a GPU since patches are independent of each other
- It works directly with the Harris corner detector feature map

Part 1: Harris corner detector

- **det(A)** measures how much intensity changes in **both directions** – high for corners.
- **trace(A)** is the sum of intensity changes along x and y – large if there's strong change in at least one direction (edge or corner).
- **$R = \det(A) - \alpha \cdot \text{trace}^2$**:
- Rewards points with strong variation in both directions →**true corners**
- Penalizes points with strong change in only one direction →**edges** or flat regions
- High R →**corner**, Low/negative R →**edge or flat**

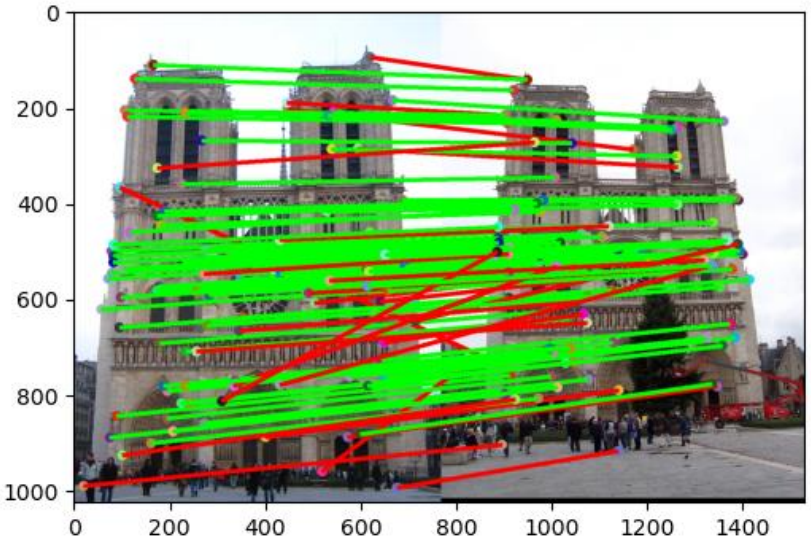
The Harris detector measures how intensity changes in different directions. The structure tensor has two eigenvalues that capture variation along two perpendicular directions. If both eigenvalues are large, the region has strong changes in two directions and is a corner. If only one is large, it is an edge, and if both are small, it is a flat region.

Part 2: Normalized patch feature descriptor

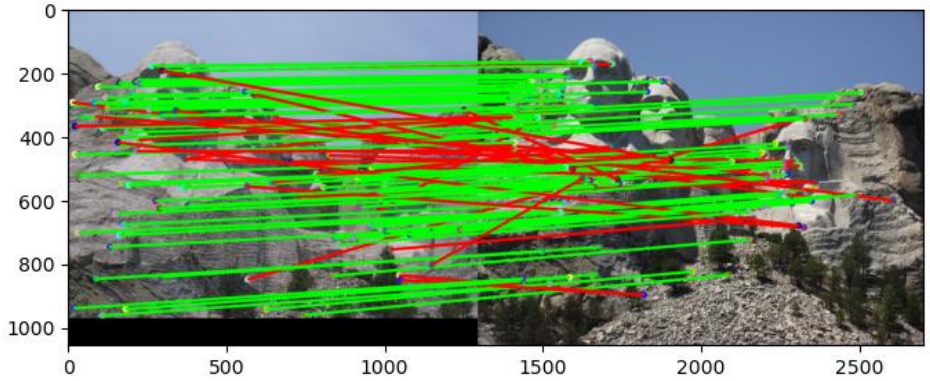


Normalized patches are not very good feature descriptors because they are fragile. Even small shifts or misalignments in the image can change the patch a lot, making matches unreliable. They are also sensitive to illumination changes and noise, and they do not handle differences in scale or rotation. Since they rely on raw pixel values, they fail to capture higher-level structure needed for robust matching.

Part 2: Feature matching

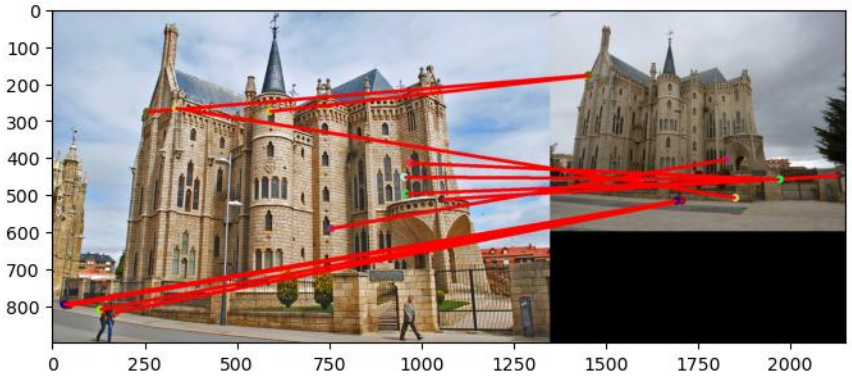


matches (out of 100): 107
Accuracy: 0.766



matches: 107
Accuracy: 0.729

Part 2: Feature matching



matches: 12

Accuracy: 0.00

The function applies the ratio test to filter matches by comparing the closest and second-closest feature distances. A match is kept only if the best match is much stronger than the second-best, reducing ambiguity.

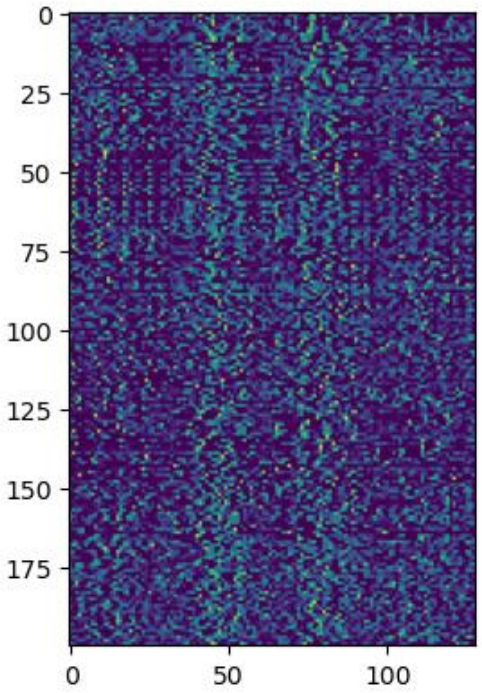
Matches are stored as index pairs between the two images, and each is given a confidence score of ***1−ratio***.

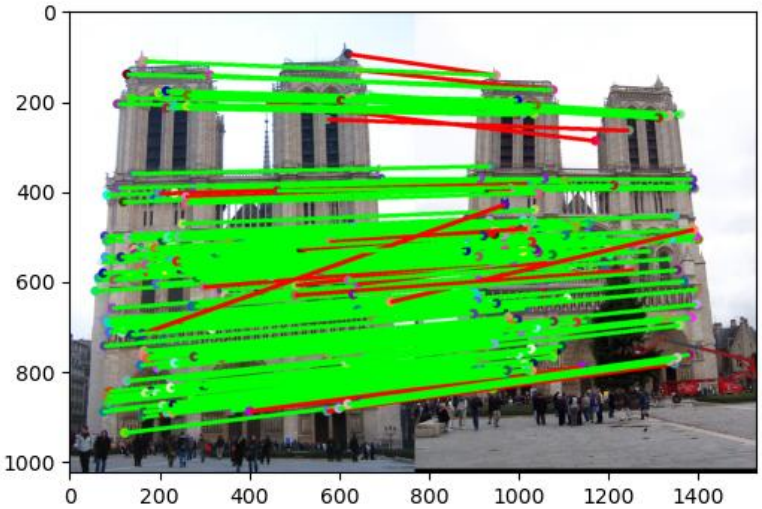
A higher confidence means the match is more reliable.

Part 2: Feature matching

Feature matches that easily pass the nearest neighbor distance ratio (NNDR) test tend to be more accurate because the best match is much closer to the query feature than the second-best match. This large gap indicates that the match is distinctive and not easily confused with other similar features. If the nearest and second-nearest distances are similar, the match is ambiguous and more likely to be incorrect. In short, a strong separation between the first and second neighbors signals a reliable and unambiguous correspondence.

Part 3: SIFT feature descriptor

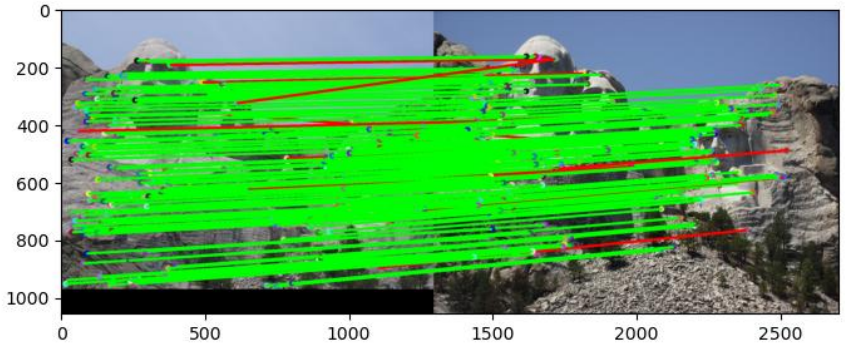




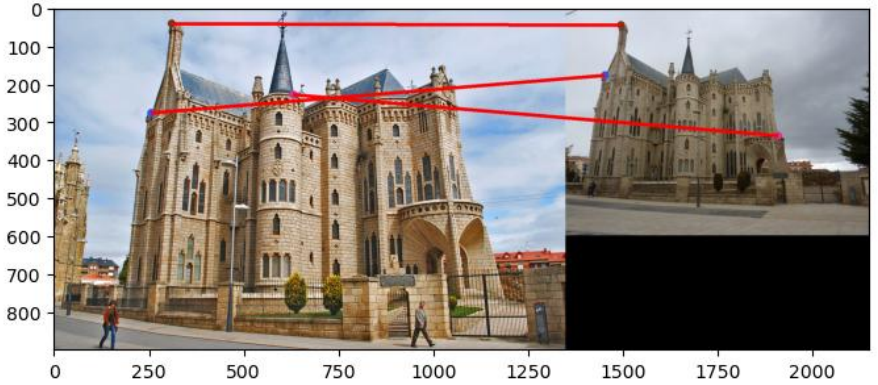
matches (out of 100): 197

Accuracy: 0.919

Part 3: SIFT feature descriptor



matches: 178
Accuracy: 0.932



matches: 3
Accuracy: 0.00

Part 3: SIFT feature descriptor

Gradient Histograms in SIFT Descriptors:

- A 16*16 window is selected around the keypoint. This window is divided into smaller cells (typically 4*4)
- For each pixel: Compute orientation and magnitude
- Each cell builds a histogram of gradient orientations, weighted by magnitude. (In the book it also mentions we do this weighting via a Gaussian distribution with the keypoint as the center)
- Histograms for all cells are concatenated to form the final descriptor
- These capture the local shape and edge directions in this window
- This is robust to small translations, rotations and illumination changes improving feature matching

SIFT features are better than normalized patches because they describe the local structure using gradient histograms instead of raw pixels. This makes them robust to small translations, rotations, and illumination changes. They are also more distinctive, allowing reliable matching across different viewpoints. Normalized patches lack this invariance and often produce ambiguous or fragile matches.

Square-rooting reduces the effect of large gradient values and emphasizes smaller ones. This makes the descriptor more robust to illumination changes and contrast changes. This might result in better matching performance

Part 3: SIFT feature descriptor

Potential Reasons for poor performance on the Gaudi pair:

1. **Scale Difference:** It is possible that due to the major scale difference in the images SIFT does not perform well since it is not scale invariant.
2. **Limited Texture/Distinctive Features:** The Gaudi images do not have a lot of high-contrast regions or distinctive features which might be another reason for the poor performance.

Part 3: SIFT feature descriptor

Our version of SIFT computes patches directly on patches around the keypoint. In case of rotation or scaling the local gradients change relative to the descriptor. This leads to poor matches.

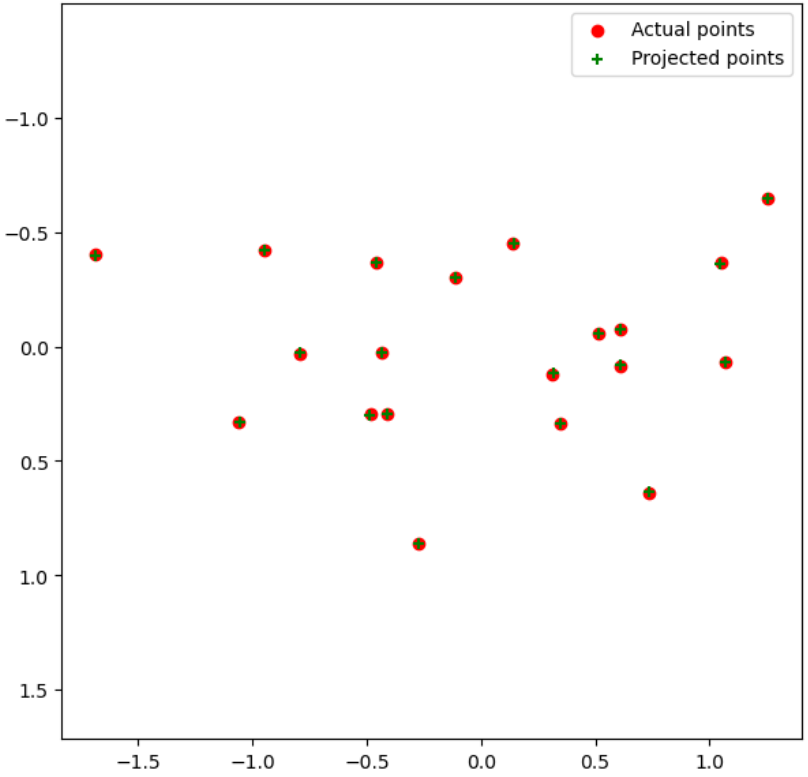
Rotation Invariance:

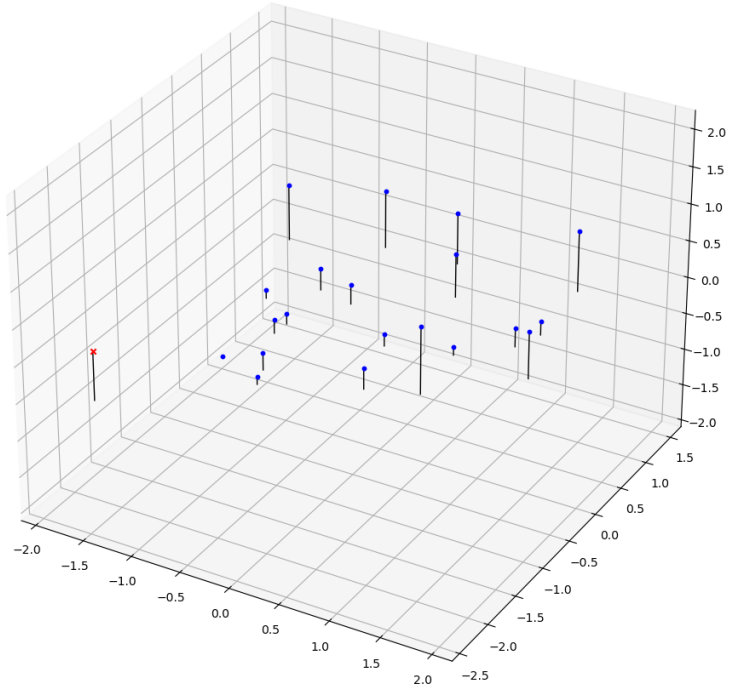
- We could possibly create multiple bins around the keypoint and detect the most dominant orientation. We assume that this is the direction in which the gradients are pointing

Scale Invariance:

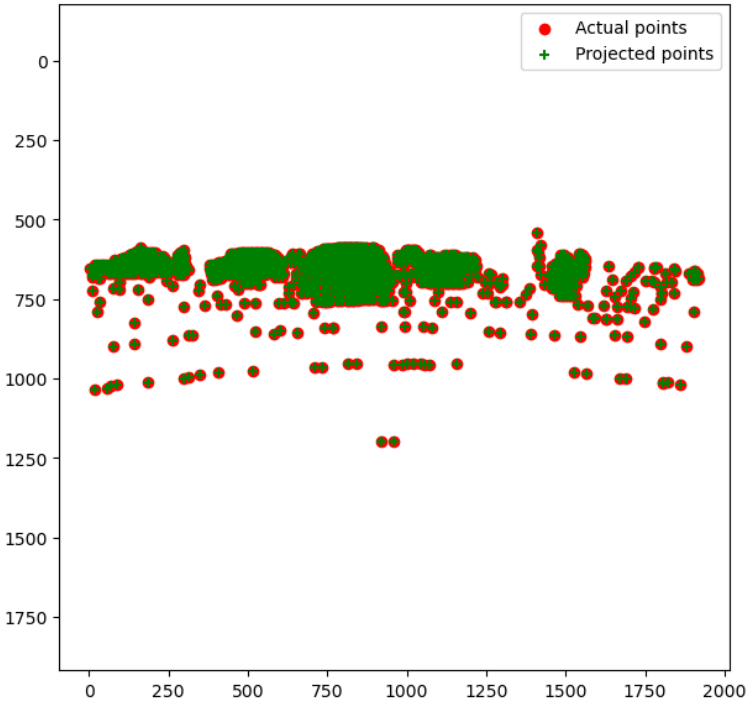
- Detect keypoints at multiple image scales (we can scale the image by downsampling) and construct a scale space pyramid

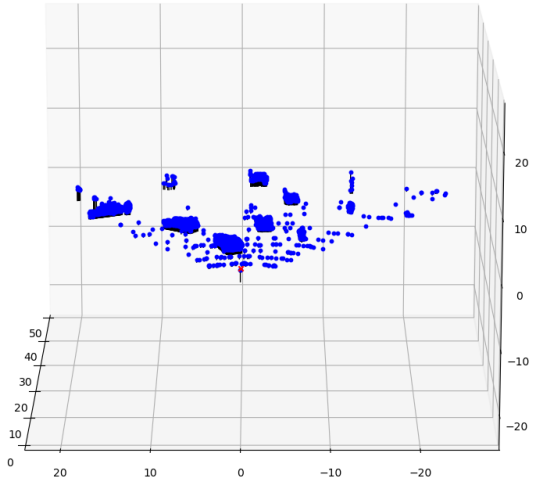
Part 4 (Extra Credit): : Projection matrix





Part 4 (Extra Credit): : Projection matrix





Part 4 (Extra Credit): : Projection matrix

A camera matrix basically relates real-world 3D coordinates to corresponding 2D pixel coordinates.

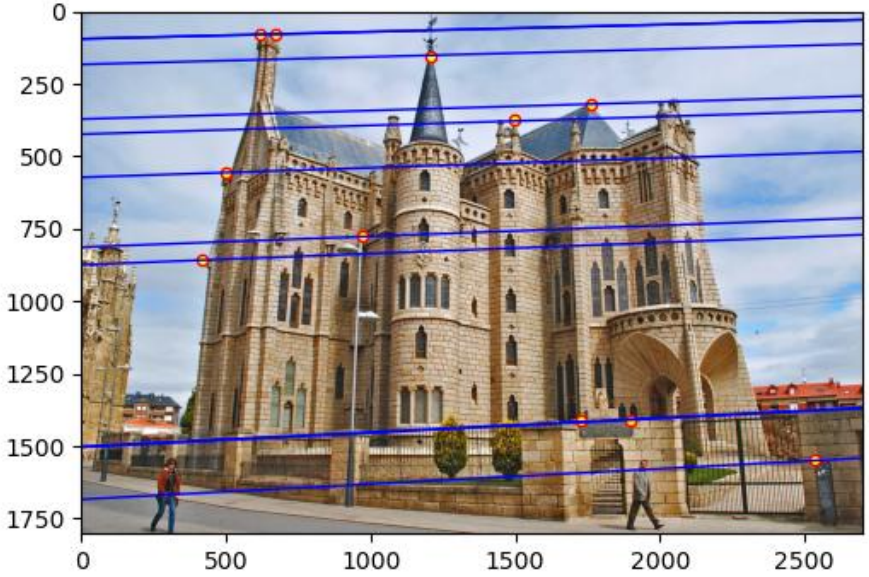
The camera matrix can be decomposed into extrinsic and intrinsic parameters.

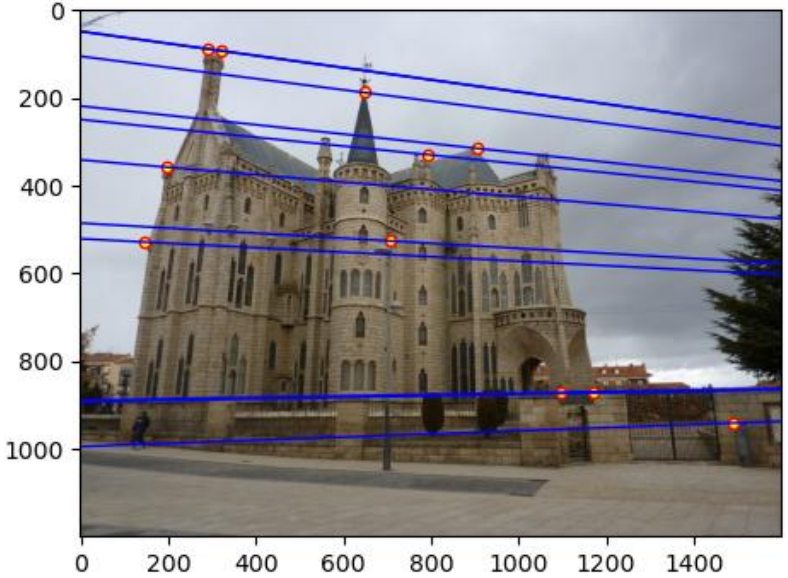
1. Intrinsic Parameters: Describe the internal properties of the camera.
2. Extrinsic Parameters: They describe the camera's position and orientation in the real-world frame of reference. This can be represented by a rotation matrix and translation vector

Decomposition = Intrinsic x [Rotation | Translation]

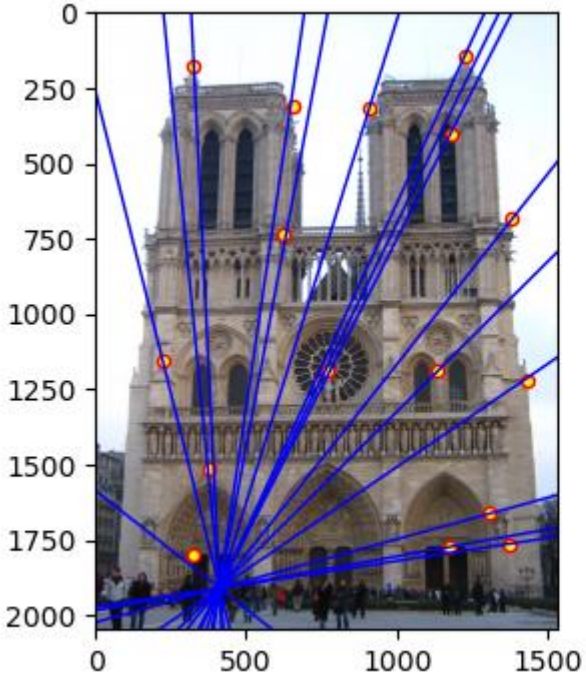
1. Focal Length: The focal length of the camera affects the intrinsic parameters of the matrix
2. Camera Orientation and Position: Affect the extrinsic parameters of the camera matrix.
3. Principal Point: This shifts where the projected point lands on the image

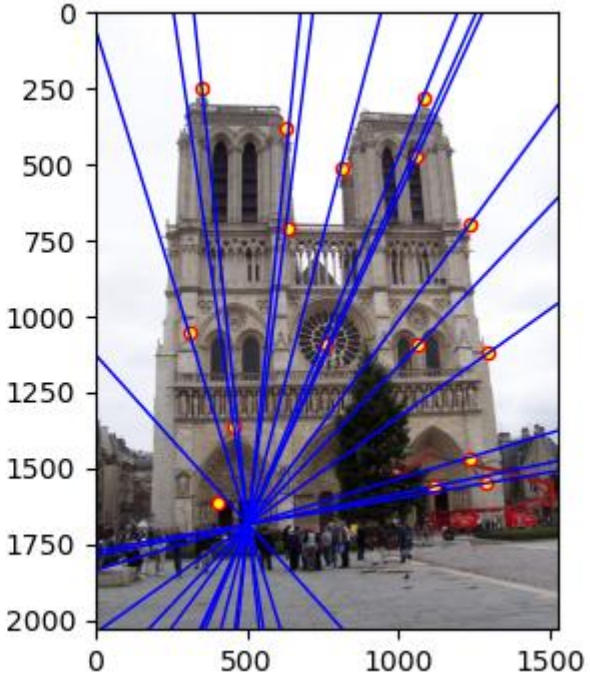
Part 5 (Extra Credit): Fundamental matrix





Part 5 (Extra Credit): Fundamental matrix





Part 5 (Extra Credit): Fundamental matrix

This is because the fundamental matrix encodes the epipolar geometry between the views. Basically, any point that is projected on to an image as a pixel, could lie anywhere along a line of possible depths when seen from another camera. The fundamental matrix maps the pixel in one image to the corresponding epipolar line in the second image (set of all possible locations where that 3D point could project)

When the camera centers project inside each other's images, the **epipoles appear inside the image boundaries**. Since every epipolar line must pass through the epipole, the lines then radiate outward from that point, creating a "starburst" pattern. This happens because the epipole is defined as the projection of the other camera's center, so if the center lies within the field of view, its projection naturally falls inside the image.

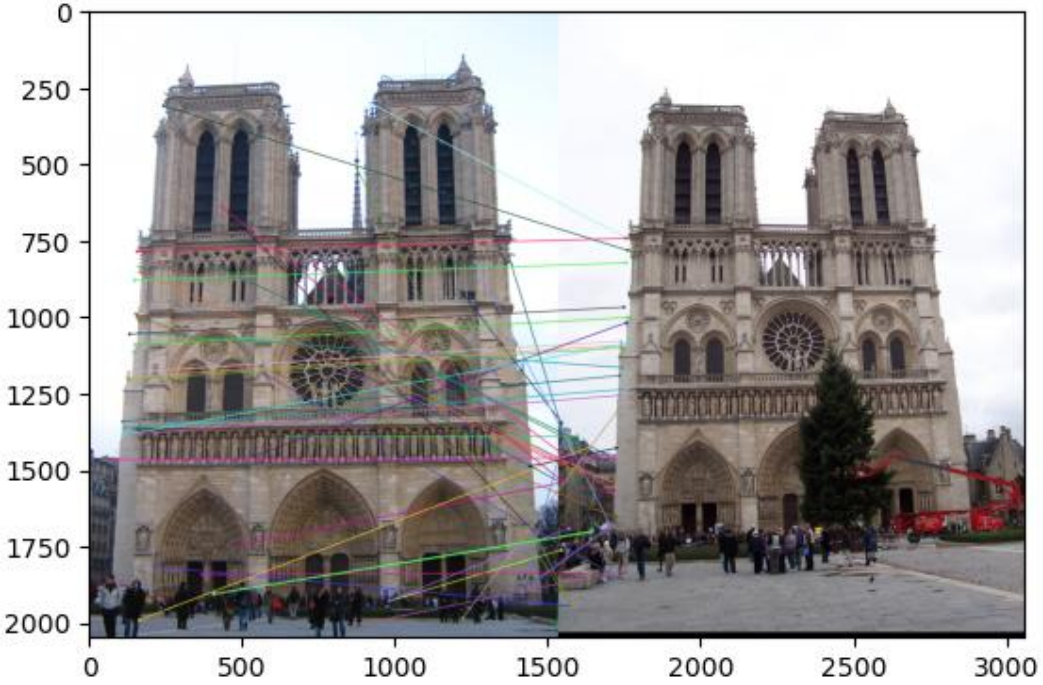
Part 5 (Extra Credit): Fundamental matrix

When epipolar lines are all horizontal it means that the camera's image planes are horizontal. This implies that there is only translation in x-direction

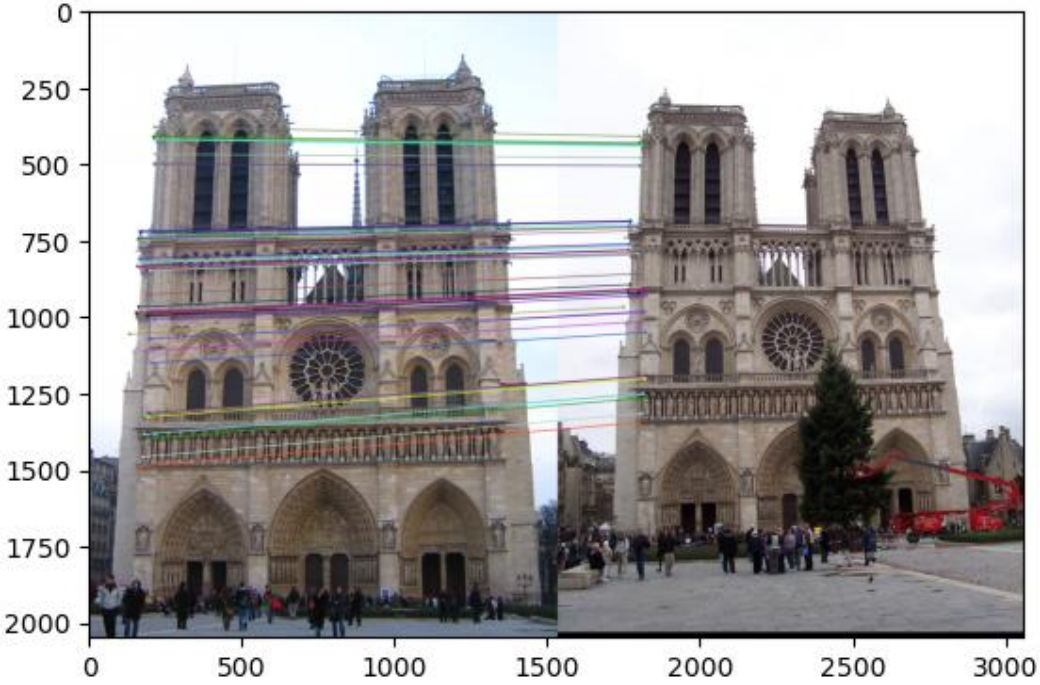
Since the fundamental matrix encodes the epipolar geometry between two images. It is defined up to a scale because multiplying all the entries by a non-zero number does not change the epipolar constraint. Only the relative values of the entries matter.

The fundamental matrix has rank 2 because it represents a mapping from points in one image to lines in the other image. For any point x in image 1 is represented by the line l = Fx in image 2. All the lines pass through the epipole so they are dependent. This forces that fundamental matrix to have rank 2 instead of rank 3.

Part 6: RANSAC



Part 6: RANSAC



Part 6: RANSAC

$$N = \frac{\log(1-p)}{\log(1-w^s)}$$

Plugging in the values, we get N = 6.471 which we can round up to 7

Plugging in the value 8 in the above sample, we get that N=12.27, which can be rounded up to 13. This is counterintuitive but can be explained because as we increase the number of samples the probability that all the points in the sample are inliers becomes lesser

We get N = 25.16, this can be rounded up to 26 iterations

Part 6: Performance comparison

Image A warped onto B using Naive Homography (NO RANSAC)

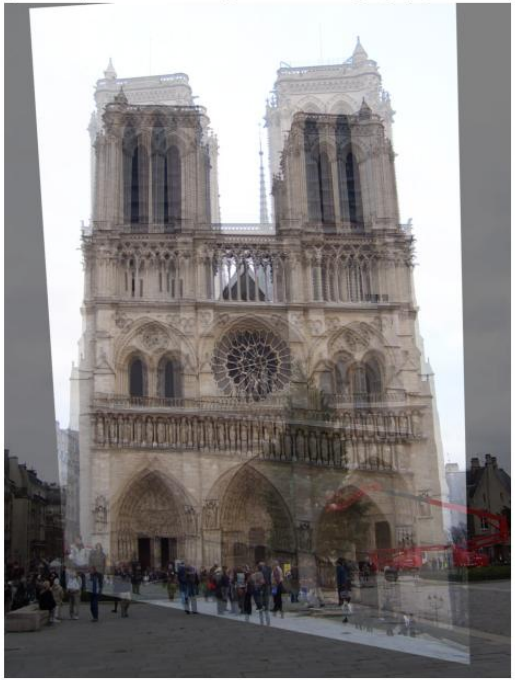
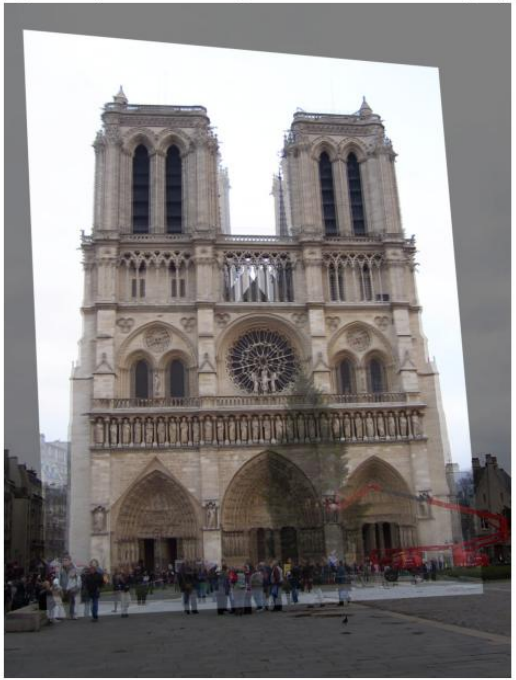


Image A warped onto B using Custom RANSAC Homography



Part 6: Performance comparison

The homography we computed using RANSAC seems much more well-aligned as compared to the naïve implementation of RANSAC.

In real-world applications RANSAC would be more effective since this kind of data always has some sort of noise due to a variety of reasons. The trade-off obviously is that RANSAC is a lot slower depending on the number of expected outliers.

These differences are due to the defect in the way least-squares operates. It is very sensitive to outliers hence a few outliers completely distorts the homography. RANSAC samples points iteratively and selects the homography that fits the largest number of inliers

Conclusion

The RANSAC-based homography estimation was significantly more effective than the naive approach because it is robust to outliers in the initial SIFT matches. In practice, SIFT matching produces some incorrect correspondences due to repetitive textures, noise, or partially visible features. Naive homography assumes that all matches are correct and computes a single transformation using all points, so even a few mismatches can distort the estimated homography and produce poor alignment.
RANSAC, in contrast, iteratively selects minimal subsets of points to compute candidate homographies and evaluates how many matches agree with each candidate (the inliers). By focusing on the subset with the most inliers, RANSAC effectively ignores the incorrect matches, producing a homography that aligns the images correctly despite noisy or mismatched points. This handling of outliers is why RANSAC provides much more accurate and reliable results in real-world scenarios.